REQUIREMENTS · v1 · 2026

Project Bootstrap

Everything Claude Code needs to pull, install, or create from the web before the first line of bot code is written.

TIME TO COMPLETE	TOTAL COST NOW	OS	STAGE
~2-3 hours	$0 (creds in hand)	macOS	pre-Phase 0

Table of Contents

Run sections in order — each one assumes the prior is done

01	**How to use this doc**	What's in / what's not.	p. 3
02	**Pre-flight checklist**	Hard prerequisites — verify before any installs.	p. 4
03	**Web accounts to create**	GitHub, Anthropic, Telegram, Alchemy.	p. 5
04	**macOS tools to install**	brew packages: uv, gh, direnv, age, etc.	p. 6
05	**Python toolchain**	uv-managed; mypy/ruff/pytest pinned.	p. 7
06	**Claude Code skills**	Drop into ~/.claude/skills/.	p. 8
07	**Built-in skills + slash commands**	What you already have; when to use it.	p. 9
08	**MCP servers**	Obsidian + research-only Polymarket MCPs.	p. 10
09	**Reference repos to clone**	py-clob-client, agents, etc.	p. 11
10	**Reference docs to read**	Polymarket / NautilusTrader / Chainlink.	p. 12
11	**Polymarket-specific**	Auth flow, gasless relayer, contracts.	p. 13
12	**Bonereaper study extras**	RPC, ML libs, Streamlit.	p. 14
13	**Deferred to Phase 5**	VPS, Lightsail, Backblaze (NOT now).	p. 15
14	**Bootstrap script**	Single bash to install most things.	p. 16
15	**Verification checklist**	Run before declaring "ready for Phase 0".	p. 18
16	**Cost summary**	What costs money now / later.	p. 19

How to use this doc

In / out scope and reading order

~2-3h	$0	16	0
Total wall-clock to complete bootstrap	Cost today (creds already in hand)	Sections — run sequentially	Lines of bot code (that comes after)

What this doc IS

- Sequential bootstrap: web accounts, local tools, Claude skills, MCPs, reference repos, docs to bookmark.
- Verification checklist that proves you're ready for RUNBOOK.md Phase 0.
- A bootstrap shell script that automates the install steps you'd otherwise type yourself.
- Costs surfaced honestly — what you pay now ($0), what you'll pay at Phase 4 ($150 Anthropic top-up), what at Phase 5 ($5/mo VPS).

What this doc IS NOT

- Architecture or strategy — that's BUILD_SPEC.pdf.
- Phase-by-phase execution — that's RUNBOOK.md.
- The bonereaper research stream — that's BONEREAPER_STUDY.pdf.
- Locked decisions — those live in memory/project_decisions.md.

Reading order: finish §02 (pre-flight) before doing anything. Then §03 → §10 in order. §14 (bootstrap script) automates §04-§09 if you'd rather not type it all by hand. §15 verifies you're ready for Phase 0.

Pre-flight checklist

Hard prerequisites — verify these BEFORE any installs

Each item is a non-negotiable. Failing any of them means stop bootstrap and resolve the underlying issue. Do not work around them.

Identity & jurisdiction

- [ ] I am physically resident in the UAE (Polymarket-permitted).
- [ ] I am NOT in US/UK/FR/BE/SG/TH/AU/Ontario.
- [ ] I will not connect a personal wallet from a restricted geography via VPN.

Hardware (laptop)

- [ ] macOS laptop with at least 16GB RAM and 80GB free disk.
- [ ] Apple Silicon (M1/M2/M3/M4) preferred for speed; Intel works.
- [ ] Stable network — fiber or 50+ Mbps cable. WS feeds need it.

Existing accounts I should already have

- [ ] GitHub account with SSH key configured.
- [ ] Anthropic Console account with API key + ≥$50 credits loaded.
- [ ] Telegram on phone (any account; we'll create a bot in §03).
- [ ] An existing exchange account where I can withdraw ~$1,050 USDC + ~$5 MATIC to Polygon (Phase 1, not now).

Tools I might already have (not blocking bootstrap)

- [ ] Homebrew installed (brew --version returns).
- [ ] Python 3.11+ available (python3 --version).
- [ ] git installed and configured with name + email.
- [ ] VS Code or Cursor (any modern editor).
- [ ] Obsidian app installed (we'll set up the vault in §04).

Web accounts to create

Sign-ups + API keys + a Telegram bot

Anthropic API

Sign up at	console.anthropic.com
What to do	Generate an API key under Settings → API Keys; load $50 credits via Settings → Plans & Billing.
Save where	.envrc as ANTHROPIC_API_KEY (do this in §05 after we install direnv).
Phase	Used Phase 0 (Telegram smoke) onward; agents start Phase 3.
Cost today	$0 if you already loaded $50 (locked decision #14).

GitHub (private repo)

Required	Personal account (free), SSH key configured, gh CLI installed (§04).
Repo creation	Done by bootstrap script in §14: gh repo create polymarket-5m-bot --private.
CI	Free GitHub Actions tier — sufficient at this scale.
Phase	0+.

Telegram bot via @BotFather

Step 1	On phone Telegram, open @BotFather → /newbot → name it (e.g. polymarket-5m-ops) → username ending in -bot.
Step 2	BotFather returns a bot token. Save.
Step 3	Message your new bot once (any text) so it has a chat with you.
Step 4	On laptop: curl "https://api.telegram.org/bot<TOKEN>/getUpdates" → find your chat.id.
Save where	.envrc as TELEGRAM_BOT_TOKEN and TELEGRAM_CHAT_ID.
Test send	curl -X POST "https://api.telegram.org/bot<TOKEN>/sendMessage" -d "chat_id=<ID>&text=hello" → confirm message arrives on phone.
Phase	0+ (smoke test in Phase 0 gate).

Alchemy (Polygon RPC)

Sign up at	alchemy.com
What to do	Create a Polygon Mainnet app → copy the WSS URL (the wss:// one, not https://).
Save where	.envrc as POLYGON_RPC_URL.
Cost	Free tier: 300M compute units/month — covers everything through Phase 6.
Phase	Used Phase 2 onward (oracle subscription + bonereaper ingest).

macOS tools to install

brew handles most of it

If you already have these, skip them. Verify with the listed check command. Bootstrap script (§14) installs anything missing.

Core tools

Tool	Purpose	Install (Homebrew)	Verify
brew	Package manager	see brew.sh	brew --version
git	Version control	brew install git	git --version
gh	GitHub CLI	brew install gh	gh auth status
uv	Python pkg/env manager	see astral.sh/uv	uv --version
node	JS runtime (some MCPs)	brew install node	node --version
direnv	Project-scoped env vars	brew install direnv	direnv --version
age	Encrypt wallet keys offline	brew install age	age --version
jq	JSON munging on CLI	brew install jq	jq --version
wscat	WS round-trip latency check	brew install node + npm i -g wscat	wscat --version
chrony	Clock skew (Phase 5 VPS only)	—	(N/A locally)

Recommended (not strictly required)

Tool	Why
iTerm2 + tmux	tmux session per project; survives terminal close
VS Code or Cursor	diff review, GitLens / ruff / mypy extensions
1Password CLI (op)	secrets injection at runtime (op run -- python ...)
Obsidian	memory vault on laptop
Postman or Insomnia	manual probing of Polymarket REST endpoints

install verification (run from any dir)

```
for cmd in brew git gh uv node direnv age jq wscat; do
  if command -v $cmd >/dev/null; then echo "OK   $cmd"; else echo "MISS $cmd"; fi
done
```

Python toolchain

uv-managed; lockfile committed

All Python deps managed via uv. Versions pinned in pyproject.toml; lockfile uv.lock committed to git. CI runs uv sync --frozen so prod and CI agree exactly.

Dev dependencies (Phase 0 install)

Package	Why	Phase
mypy	Strict static typing; gate from Phase 2	0+
ruff	Lint + format; gate from Phase 0	0+
pytest	Test runner	0+
pytest-asyncio	async test fixtures	2+
pytest-cov	Coverage report	2+
import-linter	Production never imports paper-mocks	2+
pre-commit	Local-side hooks (mirror CI)	0+
anthropic	Anthropic SDK for the agent ensemble	3+

Runtime dependencies (added later, per-phase)

Package	Why	Phase
eth-account	Generate wallets, sign EIP-712	1
web3	Polygon RPC interactions	1+
py-clob-client	Polymarket signing + orders	1+
websockets	Native WS client for Binance/Coinbase/Polymarket	2+
polars	Fast Parquet IO	2+
pyarrow	Parquet engine for Polars	2+
nautilus-trader	Backtest engine + Polymarket adapter	3
scikit-learn	Logistic + GBM for feature importance	3+ / study D
shap	Local feature explanations	study E
duckdb	Ad-hoc SQL on Parquet	study G
streamlit	Research dashboard	study G
plotly	Charts in dashboard + reports	study G
python-telegram-bot	(or just requests) — Telegram alerts	0+
prometheus-client	Bot metrics endpoint	5+

Claude Code skills

Drop into ~/.claude/skills/ before Phase 0

Claude skills are markdown files with optional metadata. Drop them into `~/.claude/skills/` and Claude Code reads them on session start. Pull these BEFORE Phase 0 — they encode hard-won Polymarket primitives we'd otherwise have to learn.

Polymarket/agent-skills (REQUIRED)

Source	github.com/Polymarket/agent-skills
Install	`git clone https://github.com/Polymarket/agent-skills.git ~/.claude/skills/agent-skills`
Contains	authentication.md (L1+L2), trading.md, websocket.md, gasless.md, markets.md, neg-risk.md, token-ops.md, bridging.md
Verify	`test -f ~/.claude/skills/agent-skills/gasless.md`
Phase	0 (clone), 1+ (used heavily)

Custom skill: poly-bot-skills (write yourself in Phase 0)

What	Project-specific skills covering OUR locked decisions (gasless-only, $5 flat sizing, deterministic Tier-3, etc.) so future Claude sessions don't re-litigate.
Where	`~/.claude/skills/poly-bot/`
Files	decisions.md (mirror of memory file), kill-switches.md, gasless-flow.md, sizing.md, telegram-alerts.md
Phase	0 (write), all (used)
Discipline	Update on every locked-decision change; commit alongside code.

Built-in skills + slash commands

What you already have on tap

These are part of the Claude Code runtime. Nothing to install — just know which ones matter for this project and when to use them.

Slash commands you'll use

Command	Use	Phase
/init	Seed the project's CLAUDE.md once layout is on disk	0
/security-review	Run before every phase that touches signing or fund movement	1, 5, 6
/loop	Periodic check (e.g. monitor backtest progress) — sparing use	3-4
/schedule	Recurring agent tasks — only after Phase 6 lives	7+
/clear	After drift detected mid-conversation	any
/config	Quick settings adjustments (theme, model)	any

Built-in skills relevant here

Skill	Use	Phase
update-config	Add hooks to settings.json (e.g. block git push without lint)	0+
fewer-permission-prompts	Auto-allow common read tool calls	0+
simplify	Code review pass on changed files before commit	all
anthropic-skills:pdf	Build dashboard exports / report PDFs	research
anthropic-skills:docx	Generate human-readable postmortems	research
claude-api	Building the 3-agent ensemble against Anthropic SDK	3+
design:design-handoff	Spec the Streamlit dashboard tabs	study G
init	Create the initial CLAUDE.md (=/init)	0
review	PR review on ensemble changes	3+
security-review	Branch-level security check before merge	1, 5, 6

MCP servers

Obsidian (required) + Polymarket research MCPs (optional)

MCP servers expose tools to Claude. We need exactly one (Obsidian) for the project. The Polymarket MCPs are research aids — read-only chat exploration of markets — and are **NEVER** used for live order flow.

Obsidian — REQUIRED for memory vault

Source	github.com/iansinnott/obsidian-claude-code-mcp
Install	Within Obsidian: Community Plugins → Browse → search 'Claude Code MCP' → Install. Then Settings → Claude Code MCP → Enable + copy URL.
Wire to Claude Code	Add to ~/.claude/settings.json under mcpServers. Exact JSON in §14 bootstrap.
Vault location	/Users/hamad/Desktop/PolyMarket-5m/state/ (laptop-only, locked decision #18)
Phase	0+
Verify	In Claude Code: 'Read state/runbooks/README.md' returns content (not tool error).

Polymarket research MCPs (OPTIONAL — chat exploration only)

MCP	What it gives Claude in chat	Install
caiovicentino/polymarket-mcp-server	45 tools: market data, real-time WS, exposure caps. Read-only mode supported.	stdio: clone + npm i + add to ~/.claude/settings.json
artvandelay/polymarket-agents	10 research tools: odds, orderbook, spread, history.	stdio: pip install + config
berlinbra/polymarket-mcp	Lighter read-only market data MCP.	stdio: clone + config
agent-next/polymarket-paper-trader	Paper-trading sim accessible via chat. Useful Phase 3-4.	stdio: clone + config

RULE: none of these MCPs ever issue real orders. Live trading is exclusively from the bot binary on the Dublin VPS. Research MCPs are for ad-hoc chat exploration during dev only (e.g. 'what's the current spread on the active 5m market?').

Reference repos to clone

Mining-only — never imported as runtime deps

All repos go under `references/` in the project, which is *gitignored*. We read them, learn patterns, sometimes copy specific functions (with attribution), but never depend on them as runtime imports.

Polymarket — official

Repo	Why we keep it local
Polymarket/py-clob-client	Auth flow + signing primitives — we DO use this as a runtime dep, but mining the README + source for patterns is invaluable.
Polymarket/rs-clob-client	Rust client, fastest. Reference for performance-critical patterns. Not used as dep.
Polymarket/agents	TS autonomous-agent framework. Mine for design patterns.
Polymarket/poly-market-maker	Reference market-maker keeper bot.
Polymarket/agent-skills	Already cloned into ~/.claude/skills/ — leave that copy alone; clone again into references/ for code-level grep if needed.

Reference 5m bot implementations

Repo	Why
Archetapp gist 7680adabc48f812a561ca79d73cbac69	Best in-the-wild 5m bot. Window/timing logic gold.
aulekator/Polymarket-BTC-15-Minute-Trading-Bot	Production-grade 15m bot — adapt sizing/risk patterns.
ThinkEnigmatic/polymarket-bot-arena	Multi-strategy arena pattern — Phase 7.
evan-kolberg/prediction-market-backtesting	NautilusTrader Polymarket adapter — fork in Phase 3.

clone all into references/ (idempotent — re-runnable)

```
mkdir -p references && cd references
for repo in \
  Polymarket/py-clob-client Polymarket/rs-clob-client \
  Polymarket/agents Polymarket/poly-market-maker \
  evan-kolberg/prediction-market-backtesting \
  aulekator/Polymarket-BTC-15-Minute-Trading-Bot \
  ThinkEnigmatic/polymarket-bot-arena ; do
    [ -d "$(basename $repo)" ] || git clone --depth 1 https://github.com/$repo.git
done
[ -d archetapp-5m-bot ] || git clone --depth 1 \
    https://gist.github.com/Archetapp/7680adabc48f812a561ca79d73cbac69 archetapp-5m-bot
cd ..
```

Reference docs to read / bookmark

Verified canonical URLs — bookmark to the project profile

Polymarket

Developer Docs	docs.polymarket.com/developers/
Auth flow (L1 + L2)	docs.polymarket.com/developers/CLOB/authentication
WebSocket subscriptions	docs.polymarket.com/developers/CLOB/websocket
Maker Rebates Program	help.polymarket.com/en/articles/13364471
ToS (verify UAE permitted)	polymarket.com/tos

On-chain primitives (Polygon)

Chainlink BTC/USD on Polygon	data.chain.link/feeds/polygon/mainnet/btc-usd
Polygon block explorer	polygonscan.com
USDC.e → native USDC migration tracking	theblock.co/post/388670

Trading engine + ML libs

NautilusTrader Polymarket integration	nautilustrader.io/docs/latest/integrations/polymarket/
scikit-learn user guide	scikit-learn.org/stable/user_guide.html
Polars docs	pola.rs
DuckDB docs	duckdb.org/docs/
Streamlit docs	docs.streamlit.io

Anthropic

Anthropic API docs	docs.anthropic.com
Prompt caching guide	docs.anthropic.com/en/docs/build-with-claude/prompt-caching
Models & pricing	anthropic.com/pricing
Rate-limit tiers	docs.anthropic.com/en/api/rate-limits

Polymarket-specific

Auth, gasless, contract addresses

Endpoints (verify before coding)

Use	URL / address
REST API base	https://clob.polymarket.com
WebSocket — markets	wss://ws-subscriptions-clob.polymarket.com/ws/market
WebSocket — user events	wss://ws-subscriptions-clob.polymarket.com/ws/user
Gamma API (markets list)	https://gamma-api.polymarket.com
Polygon chain ID	137

Auth flow (memorize)

Step	What
L1	EIP-712 signature with the Signer wallet's private key — produces apiKey + secret + passphrase. One-time.
L2	HMAC-SHA256 of (timestamp + method + path + body) using the L2 secret. Sent on every request via 5 POLY_* headers.
Headers	POLY_ADDRESS, POLY_SIGNATURE, POLY_TIMESTAMP, POLY_NONCE, POLY_API_KEY
Library	py-clob-client wraps both L1 and L2 — never roll your own.

Gasless relayer (locked decision #17)

Field	Value
Why	Avoid public Polygon mempool exposure (MEV / sandwiching).
Reference	~/.claude/skills/agent-skills/gasless.md (cloned in §06).
Coverage	Order placement is gasless. USDC approval to the exchange contract is NOT (one-time on-chain in Phase 1).
Phase	1.0 design doc, 1.5 first test order, 6 live.

Bonereaper study extras

Adds nothing the main bot doesn't already need + a few ML libs

The behavioral study (BONEREAPER_STUDY.pdf) reuses everything the main bot uses: Polygon RPC, captured Phase 2 archives, Anthropic API. Three extras specific to the study:

Extras

Item	Why	Phase (study)
Bonereaper wallet address	0xeebde7a0e019a63e6b476eb425505b7b3e6eba30 — verify on PolygonScan that account is active.	A
scikit-learn	Logistic regression + gradient-boosted trees for feature importance.	D, E
SHAP	Local feature explanations on the dashboard.	E, G
DuckDB	Ad-hoc SQL on labelled.parquet — drives the dashboard.	G
Streamlit + Plotly + Polars	Research dashboard stack.	G
Pre-flight audit script	scripts/research/quick_audit.py — runs against Alchemy free tier.	A

Hard abort gate (Phase A): if bonereaper's actual on-chain 30-day rolling win-rate is below 60% OR observable trades < 500 OR volume drop > 40% in last 30 days — abort the entire study. Don't anchor weights to a degrading source. *This audit happens before any other study work.*

Deferred to Phase 5 (DON'T do now)

VPS, observability, backups — not part of bootstrap

Mentioned here only so you don't try to set them up early. Each is provisioned in Phase 5 with explicit gates.

Phase 5+ services

Service	When	Cost
AWS Lightsail eu-west-1 Dublin	Phase 5 — sub-5ms VPS	~$5/mo
Grafana Cloud free tier	Phase 5 — warn/info dashboards	$0
Backblaze B2	Phase 5 — trades.db offsite backup	~$0.005/GB/mo
Hardware wallet (Ledger Nano S)	Phase 7 step 7.1 ($3k bankroll)	~$80 one-time
PagerDuty / SMS escalation	Phase 7+ at $1k+ bankroll	depends on tier
Frankfurt warm-standby VPS	Phase 8 — DR	~$5/mo

Why deferred: none of these are needed for laptop-only Phases 0-4. Provisioning early adds cost, attack surface, and lifecycle headache before there's anything to host.

Bootstrap script

One bash to install most of the local stack

Write to `scripts/bootstrap.sh`, `chmod +x`, run from project root. Idempotent — re-runs are safe. Does not create any web accounts (those need browser); does not load Anthropic credits (you do that on console.anthropic.com).

scripts/bootstrap.sh — full content (1/2)

```
#!/usr/bin/env bash
set -euo pipefail

echo "== Polymarket BTC 5-Min Bot — bootstrap =="

# 1. brew + brew packages
command -v brew >/dev/null || { echo "Install Homebrew: https://brew.sh"; exit 1; }
for p in git gh node direnv age jq; do
  brew list $p >/dev/null 2>&1 || brew install $p
done
command -v wscat >/dev/null || npm install -g wscat

# 2. uv (Python pkg manager)
command -v uv >/dev/null || curl -LsSf https://astral.sh/uv/install.sh | sh

# 3. project skeleton
uv init --python 3.11 --package 2>/dev/null || true
uv add --dev mypy ruff pytest pytest-asyncio pytest-cov import-linter pre-commit anthropic
mkdir -p src/polymarket_bot tests design references state config
mkdir -p runbooks/incidents runbooks/gates runbooks/postmortems runbooks/scripts
mkdir -p state/runbooks state/postmortems state/research/bonereaper
touch src/polymarket_bot/__init__.py tests/__init__.py
```

scripts/bootstrap.sh — full content (2/2)

```
# 4. drop Polymarket/agent-skills into ~/.claude/skills/
mkdir -p ~/.claude/skills
[ -d ~/.claude/skills/agent-skills ] || \
  git clone https://github.com/Polymarket/agent-skills.git ~/.claude/skills/agent-skills

# 5. clone reference repos
mkdir -p references && pushd references >/dev/null
for repo in Polymarket/py-clob-client Polymarket/rs-clob-client \
            Polymarket/agents Polymarket/poly-market-maker \
            evan-kolberg/prediction-market-backtesting \
            aulekator/Polymarket-BTC-15-Minute-Trading-Bot \
            ThinkEnigmatic/polymarket-bot-arena ; do
  [ -d "$(basename $repo)" ] || git clone --depth 1 https://github.com/$repo.git
done
[ -d archetapp-5m-bot ] || git clone --depth 1 \
  https://gist.github.com/Archetapp/7680adabc48f812a561ca79d73cbac69 archetapp-5m-bot
popd >/dev/null

# 6. .gitignore protects secrets and big files
cat > .gitignore <<'EOF'
.envrc
*.age
references/
state/
runbooks/gate-evidence/
ticks/
trades.db
*.db-shm
*.db-wal
EOF

# 7. .envrc.example (committed) and reminder to fill .envrc
cat > .envrc.example <<'EOF'
export ANTHROPIC_API_KEY="sk-ant-..."
export TELEGRAM_BOT_TOKEN="..."
export TELEGRAM_CHAT_ID="..."
export POLYGON_RPC_URL=""
export POLY_PRIVATE_KEY=""
export POLY_API_KEY=""
export POLY_API_SECRET=""
export POLY_API_PASSPHRASE=""
export POLY_FUNDER_ADDRESS=""
export POLY_SIGNATURE_TYPE="1"
EOF

# 8. Done
echo ""
echo "== bootstrap complete =="
echo "NEXT: cp .envrc.example .envrc; fill in real values; direnv allow"
echo "THEN: review § 15 verification checklist before Phase 0"
```

Verification checklist

Run before declaring "ready for Phase 0"

Tick every box. If anything fails, fix the underlying issue and re-tick. Don't start Phase 0 with red checks.

System tools

- [ ] command -v brew git gh uv node direnv age jq wscat → all OK
- [ ] python3 --version → 3.11 or higher
- [ ] node --version → v20 or higher
- [ ] uv run python -c 'import sys; assert sys.version_info >= (3,11)' → exits 0

Web accounts ready

- [ ] GitHub: gh auth status → authenticated
- [ ] Anthropic: ANTHROPIC_API_KEY exported and starts with sk-ant-
- [ ] Telegram: smoke message arrives on phone via curl
- [ ] Alchemy: POLYGON_RPC_URL set and starts with wss://

Project skeleton on disk

- [ ] ls pyproject.toml uv.lock → both exist
- [ ] ls -la state/ references/ → both exist (gitignored)
- [ ] test -f ~/.claude/skills/agent-skills/gasless.md → exists
- [ ] test -d references/py-clob-client → exists
- [ ] make verify → exits 0 (mypy + ruff + pytest all green on empty src)

Documents read at least once

- [ ] Read MASTERPLAN.md end-to-end (rationale).
- [ ] Read RUNBOOK.md sections A, B, D (Phase 0 has all the gates).
- [ ] Read BUILD_SPEC.pdf (skim — 24 pages).
- [ ] Skim BONEREAPER_STUDY.pdf — know the abort gate (Phase A win-rate ≥ 60%).
- [ ] memory/project_decisions.md — read all 18 locked decisions.

Vault + skills wired

- [ ] Obsidian vault opens at /Users/hamad/Desktop/PolyMarket-5m/state/
- [ ] obsidian-claude-code-mcp plugin enabled in Obsidian.
- [ ] Claude Code can read state/runbooks/README.md (test in a session).

Cost summary

What costs money now / later

Today (bootstrap)

Item	Cost
Anthropic API credits (already loaded)	$50 (decision #14)
Brew packages, GitHub, Telegram, Alchemy free tier	$0
Reference repos (clones)	$0
Obsidian + plugin	$0
Total bootstrap cost	**$0 incremental**

Per-phase forward look

Phase	Cost	Driver
Phase 0 — Foundations	$0	—
Phase 1 — Wallet & Auth	~$5	Polygon gas + small USDC test
Phase 2 — Data Plane	~$5	Alchemy free tier covers most
Phase 3 — Backtest A/B	$15-50	Anthropic API (cache-disciplined)
Top up before Phase 4	+$100 (to ~$150)	Anthropic 14d × 24/7 paper
Phase 5 — VPS	$5/mo recurring	Lightsail Dublin
Top up before Phase 6	+$150 (to ~$300)	Anthropic 14d × 24/7 live
Phase 6 — Live	$1,000 capital	Working capital — at risk, not consumed
Phase 7 — Scale ($3k)	+$80 one-time	Hardware wallet (Ledger)

Sustained ops (post Phase 6): ~$200/month — primarily Anthropic API at empirical $0.19-0.58/hour (decision #14). Cache discipline can cut this in half.

$0
Bootstrap cost today

~$170
Setup spend through Phase 5

$1,000
Working capital at Phase 6

$200/mo
Sustained OPEX

When this checklist is fully ticked, you are ready for RUNBOOK.md Phase 0. Open the runbook, paste task 0.1 verbatim into Claude Code as your first prompt, and start.